

GraphPoint: Semantic Entity Graphs and Point Trajectories for Compositional Robot Manipulation

Kang Luo, Hesheng Wang[†]

Abstract—Compositional manipulation requires robots to reuse familiar entities and behaviors under new instructions, both within an atomic task and across a sequence of tasks. We introduce CoMani, a benchmark that isolates modifier, action-type, and sequential composition through controlled task splits. We then present GraphPoint, a policy system that represents observations as semantic entity graphs and actions as future gripper point trajectories. The graph separates actor, patient, and target geometry, with role-conditioned local encoding and action- and modifier-conditioned interaction along an actor–patient–target chain. Coordinates relative to the tool center point and role-aware point collapse encourage reuse of geometric relationships across task combinations. A conditional flow decoder generates point trajectories, while a learned progress head enables subtask switching without sequence-level demonstrations. GraphPoint achieves 75.6% and 64.4% success on held-out modifier and action combinations, respectively. On 3-step sequences, it achieves 51.7% success with environment-based switching and 38.3% with self-switching. Ablations identify relative coordinates, role structure, and geometric augmentation as key contributors to compositional generalization.

I. INTRODUCTION

A robot that can put a bottle on a stove should be able to interpret placing it to the right of that stove as a new combination of familiar concepts. It should also be able to combine this behavior with placing a bowl and moving a package of cheese, without requiring demonstrations of every complete sequence. These capabilities involve composition at 2 levels: within a subtask, where entities, actions, and modifiers determine the desired interaction, and between subtasks, where individual behaviors must execute in order from the states left by preceding actions.

Imitation learning has made substantial progress through action chunking [1], generative action prediction [2], and large-scale vision-language-action pretraining [3]. Point-based policies further provide geometric representations for observations and actions [4], [5], [6]. However, geometry alone does not specify what an entity should do in a task. The same scene may require different motions depending on the instructed action or relation, and success on individual tasks does not establish that a policy can compose them into longer instructions. This motivates an explicit interface between task semantics, entity geometry, and action generation.

We introduce GraphPoint, which represents an active subtask as a graph of actor, patient, and target point entities. These roles identify the gripper, the manipulated object,

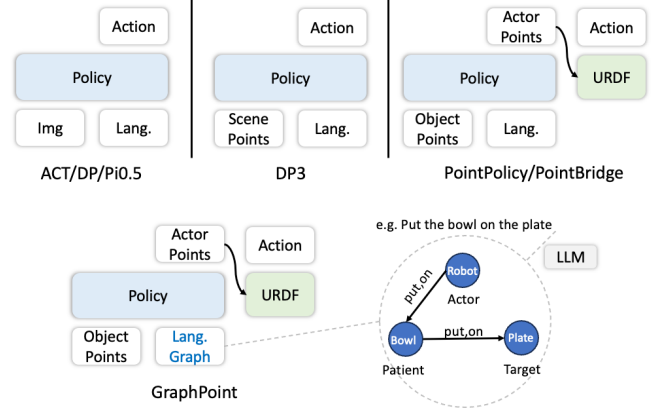


Fig. 1. Policy representation interfaces. GraphPoint organizes entity points into a language-conditioned role graph and converts predicted actor points into robot actions through gripper geometry.

and the reference entity, respectively. A shared encoder processes each entity locally and exchanges temporal summaries through an actor–patient–target chain. Role embeddings condition local geometry encoding, while separately encoded action types and modifiers condition interaction and trajectory decoding. The policy predicts future actor points in the same tool center point (TCP)-relative space as its observations, and geometric alignment converts these predictions into robot poses. Role-aware point collapse augments object geometry during training, and a progress head supports reuse of the policy across subtasks. Fig. 1 contrasts this interface with image-, scene-point-, and entity-point-based policies.

To isolate these capabilities, we introduce CoMani, a LIBERO-based benchmark [7] with suites for modifier, action-type, and sequential composition. Matched initial scenes reduce visual task shortcuts, and atomic-only training separates skill acquisition from sequence execution. Our contributions are:

- CoMani, a benchmark addressing the need to distinguish composition of modifiers, action types, and atomic skills through controlled in-distribution (ID) and out-of-distribution (OOD) splits over 18 modifier tasks, 10 action tasks, and 6 sequences.
- GraphPoint, a framework coupling semantic entity graphs with task-conditioned point-trajectory control and progress-based subtask switching, validated through baseline comparisons and component ablations on CoMani.

Kang Luo and Hesheng Wang are with IRMV Lab, the Department of Automation, Shanghai Jiao Tong University.

[†]Corresponding author email: wanghesheng@sjtu.edu.cn

II. RELATED WORK

Visual and point-based policies. ACT [1] predicts action chunks, DP [2] generates actions through conditional denoising, and $\pi_{0.5}$ [3] uses large-scale vision-language-action pretraining. Geometric alternatives include DP3’s sparse scene point clouds [4], PointPolicy’s semantic observation and action keypoints learned from human videos [5], and PointBridge’s unified points for synthetic-data training and sim-to-real transfer [6]. GraphPoint organizes task-relevant points by semantic role and conditions their interaction on action types and modifiers.

Object-centric and graph representations. GROOT [8] uses segmented object point clouds and segmentation correspondence for visual generalization. GraphMimic [9] predicts object and visual-action graphs from videos to condition control. Compose by Focus [10] encodes task-relevant subgraphs and combines diffusion-based atomic skills with a vision-language planner. Building on these established object-centric and graph-based approaches, GraphPoint couples actor/patient/target roles with explicit action/modifier conditioning and point-trajectory generation.

Compositional generalization. CLIPort [11] combines semantic features with spatial action prediction; VIMA [12] evaluates multimodal prompting across four generalization levels. CompoSuite [13] holds out combinations of robots, objects, obstacles, and objectives, while LIBERO [7] studies declarative and procedural transfer and CALVIN [14] evaluates language-conditioned task sequences. CoMani isolates modifier composition, action-type composition, and three-step execution in LIBERO, using matched scenes to reduce visual task shortcuts and atomic-only training to test sequential reuse.

III. METHOD

A. Overview

GraphPoint represents manipulation as interactions among 3D point entities and predicts future gripper points in the same geometric space. As shown in Fig. 2, language analysis, visual localization, and point tracking construct a task-conditioned entity graph, which a shared policy maps to a point trajectory and a subtask progress estimate. For a multi-step instruction, the system instantiates this graph for each subtask and uses predicted progress to advance through the sequence. The learned policy receives entity geometry and structured language conditions; color images are used by the perception front end.

B. Task-to-Graph Construction

Semantic decomposition. A language model decomposes an instruction into ordered subtasks, each specified by an action type, a modifier, and entity roles. The *actor* A denotes the gripper, the *patient* P denotes the manipulated object or part, and the *target* T denotes the reference entity. For example, “put the cheese to the right of the stove” assigns cheese to P , stove to T , *put* to the action type, and *right* to the modifier. Entity names guide visual grounding, whereas action types and modifiers are encoded separately using

frozen BGE embeddings [15] and learned projections. We denote their concatenation by the task condition c and the learned embedding of role r by e_r . For readability, c also denotes the version with separately normalized action and modifier embeddings used by the prediction heads. Absent modifiers use a learned null embedding, and absent entities are masked.

Grounded point entities. A pretrained vision-language locator identifies the requested objects or parts, and SAM 2 [16] produces masks from the localization prompts. We sample 32 points per visual entity and track their image coordinates with TAPNext++ [17]. Depth and camera intrinsics back-project these tracks into 3D. The actor instead uses 6 keypoints computed from robot geometry and proprioception: the gripper root, TCP, 2 finger bases, and 2 fingertips. We align all entities in camera coordinates and subtract the current TCP position from every point in the observation history and prediction horizon. Thus, coordinates retain camera-axis orientation while sharing a translation origin at the current TCP. This representation reduces dependence on absolute scene placement. Actor points are padded with a validity mask, and coordinates are normalized using training-set quantiles. The policy observes 10 frames, including the current frame.

C. Role-Structured Entity Graph Encoder

Role-conditioned geometry and task-conditioned interaction. A shared point MLP embeds entity coordinates, with keypoint identity embeddings added for the actor. Each entity at each observed time receives 4 learned CLS tokens to summarize its geometry. The encoder alternates local encoding within each entity and global interaction among entity summaries:

$$\begin{aligned} (\tilde{C}_r, F_r^{\ell+1}) &= \text{Local}_\ell(C_r^\ell, F_r^\ell; e_r), \\ C^{\ell+1} &= \text{Graph}_\ell(\{\tilde{C}_r\}_{r \in \{A, P, T\}}; c). \end{aligned} \quad (1)$$

Here, F_r and C_r denote point features and CLS summaries, respectively, with observation-time indices omitted for clarity. Local self-attention processes each entity and frame independently, conditioned on its role e_r . Graph attention exchanges only CLS summaries across entities and observed times, conditioned on the action and modifier c . Its connectivity follows the bidirectional chain $A \leftrightarrow P \leftrightarrow T$, with additional within-role temporal connections. Direct actor–target attention is masked, so their interaction is mediated by the patient through successive layers. This structure preserves separate entity summaries and encodes gripper–object and object–reference interactions. Temporal rotary embeddings distinguish observation times. After 8 layers, the current-time CLS summaries form memory M_t , incorporating both geometry and interaction history.

Semantic injection. Both local and graph blocks contain attention and feed-forward sublayers, whose inputs are modulated through AdaLN:

$$\text{AdaLN}(h; q) = (1 + \gamma(q)) \odot \text{LN}(h) + \beta(q). \quad (2)$$

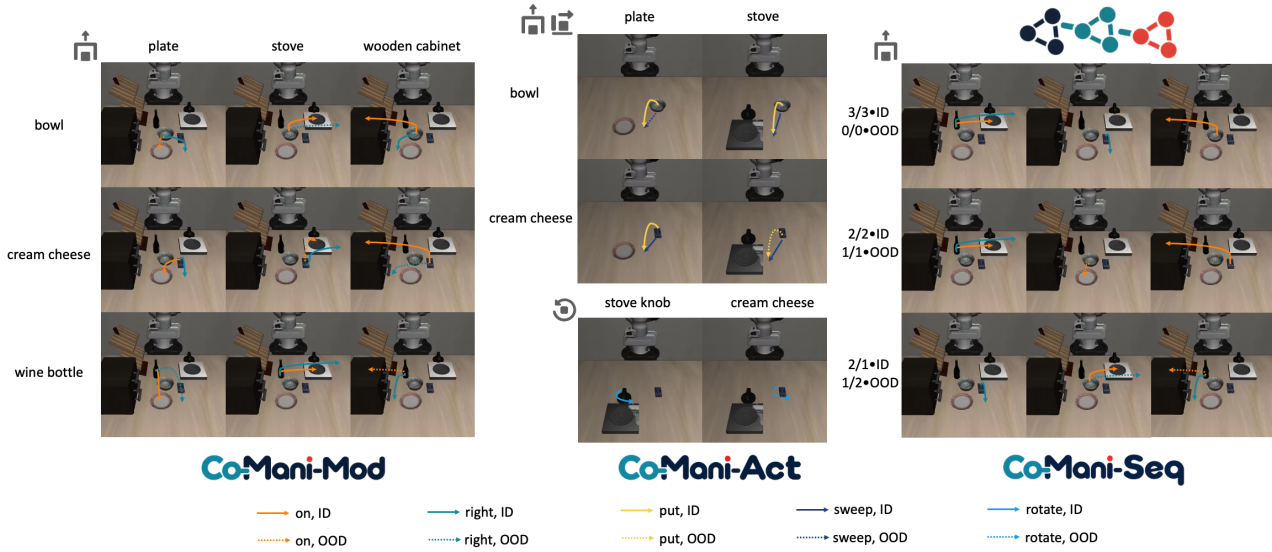


Fig. 3. CoMani task construction and splits. Left: CoMani-Mod varies modifiers with a fixed put action. Center: CoMani-Act varies action types with a fixed right modifier. Right: CoMani-Seq composes atomic tasks from CoMani-Mod into 6 sequences of 3 steps, shown in 3 rows with execution proceeding from left to right.

E. Subtask Progress and Long-Horizon Composition

The progress head estimates how far the active subtask has advanced from the current patient and target summaries:

$$\hat{s}_t = \text{Progress}_\theta(C_{t,P}, C_{t,T}; c). \quad (6)$$

It uses a task-conditioned MLP with AdaLN and a sigmoid output, so the same geometry is evaluated under the active action and modifier. The readout excludes direct actor summaries, although graph attention can propagate actor information into patient and target summaries. Training targets increase linearly from 0 to 1 within each annotated subtask interval, and the complete objective is

$$\mathcal{L} = \mathcal{L}_{\text{flow}} + \text{SmoothL1}(\hat{s}_t, s_t). \quad (7)$$

At deployment, the planner advances when predicted progress exceeds a threshold for a consecutive window of policy calls. We denote this learned transition rule as self-switching (SS). Environment-based switching (ES) instead uses an environment-provided subtask completion signal to separate execution performance from learned transition timing. On a transition, the system releases and lifts the gripper, selects the next subtask’s entities and language conditions, and resets policy history. The same atomic policy is reused throughout the sequence.

IV. COMANI BENCHMARK

CoMani comprises 3 suites testing modifier, action-type, and sequential composition, illustrated in Fig. 3 and listed in Table I. We construct the dataset with 50 trajectories for each atomic task, generated using MimicGen [19]. Within each suite, initial scenes are matched wherever feasible to require instruction-dependent behavior and discourage fixed vision-to-action mappings. We evaluate each split over 30 trials per atomic task and 10 per sequence.

CoMani-Mod. With the pick-and-place action *put* fixed, this suite combines 3 objects, 3 targets, and the modifiers *on* and *right* into 18 tasks. The split contains 12 in-distribution tasks, denoted ID, and 6 out-of-distribution tasks, denoted OOD. Each OOD object–target pair appears in the ID split with the opposite modifier, isolating generalization to unseen combinations of familiar factors. The ID/OOD comparison therefore tests whether a policy can apply the instructed relation to a familiar object–target pair beyond its demonstrated placement.

CoMani-Act. With *right* fixed, this suite varies *put*, *sweep*, and *rotate* across 10 tasks: 7 ID and 3 OOD. For *put*/*sweep*, each OOD object–target pair appears in the ID split with the other action; for rotation, the ID task uses the stove knob and the OOD task uses cream cheese. These tests examine whether a policy can reuse an action with familiar entities in a new combination and transfer rotation across different manipulated entities. *Put*/*sweep* pairs share initial states and final-position goals, but success additionally requires the execution to exhibit the instructed action. CoMani-Act therefore evaluates both goal attainment and whether the manipulation process follows the specified *put*, *sweep*, or *rotate* behavior.

CoMani-Seq. This suite tests whether policies learned from CoMani-Mod atomic demonstrations can execute 3-step instructions without sequence-level demonstrations. It contains 6 sequences organized into 3 fixed-order pairs, each varying one modifier between *on* and *right of*, without permuting subtasks. The 3 pairs contain 3 ID, 2 ID + 1 OOD, and 2/1 ID + 1/2 OOD atomic tasks, respectively, as shown on the right of Fig. 3. The all-ID pair tests composition and transitions among familiar skills; the remaining pairs test whether execution also generalizes to held-out atomic combinations, with the last pair increasing the OOD count

TABLE I
CoMANI TASK LISTS AND ID/OOD SPLITS. IN (C), ID/OOD GIVES ATOMIC TASK COUNTS; ATOMIC IDs REFER TO (A), WITH OOD IDs UNDERLINED.

(a) CoMani-Mod			(b) CoMani-Act		
ID	Task instruction	Split	ID	Task instruction	Split
0	Put the bowl on the stove.	ID	0	Put the bowl to the right of the plate.	ID
1	Put the bowl on the wooden cabinet.	ID	1	Put the bowl to the right of the stove.	ID
2	Put the bowl to the right of the plate.	ID	2	Put the cream cheese to the right of the plate.	ID
3	Put the bowl to the right of the wooden cabinet.	ID	3	Rotate the stove knob right.	ID
4	Put the cream cheese on the plate.	ID	4	Sweep the bowl to the right of the stove.	ID
5	Put the cream cheese on the wooden cabinet.	ID	5	Sweep the cream cheese to the right of the plate.	ID
6	Put the cream cheese to the right of the plate.	ID	6	Sweep the cream cheese to the right of the stove.	ID
7	Put the cream cheese to the right of the stove.	ID	7	Put the cream cheese to the right of the stove.	OOD
8	Put the wine bottle on the plate.	ID	8	Rotate the cream cheese right.	OOD
9	Put the wine bottle on the stove.	ID	9	Sweep the bowl to the right of the plate.	OOD
10	Put the wine bottle to the right of the stove.	ID	(c) CoMani-Seq		
11	Put the wine bottle to the right of the wooden cabinet.	ID	ID	ID/OOD	Sequence
12	Put the bowl on the plate.	OOD	0	3/0	Wine bottle on stove → cream cheese right of plate → bowl on wooden cabinet
13	Put the bowl to the right of the stove.	OOD	1	3/0	Wine bottle right of stove → cream cheese right of plate → bowl on wooden cabinet
14	Put the cream cheese on the stove.	OOD	2	2/1	Wine bottle on stove → bowl on plate → cream cheese on wooden cabinet
15	Put the cream cheese to the right of the wooden cabinet.	OOD	3	2/1	Wine bottle right of stove → bowl on plate → cream cheese on wooden cabinet
16	Put the wine bottle on the wooden cabinet.	OOD	4	2/1	Cream cheese right of plate → bowl on stove → wine bottle on wooden cabinet
17	Put the wine bottle to the right of the plate.	OOD	5	1/2	Cream cheese right of plate → bowl right of stove → wine bottle on wooden cabinet
			Atomic IDs		
			9 → 6 → 1		
			10 → 6 → 1		
			9 → <u>12</u> → 5		
			10 → <u>12</u> → 5		
			6 → 0 → <u>16</u>		
			6 → <u>13</u> → <u>16</u>		

from 1 to 2 through a single modifier change.

Evaluation metrics. CoMani-Mod uses success rate (SR), the fraction of trials satisfying the instructed final-goal predicates. CoMani-Act uses *action-consistent success rate* (ACSR):

$$ACSR = \frac{100}{N} \sum_{i=1}^N g_i a_i, \quad (8)$$

where g_i indicates final-goal satisfaction and a_i indicates execution of the instructed action type. Put requires lifting, transporting, and placing the object; sweep requires pushing it along the support surface; rotate requires turning the object or knob in the instructed direction. Reaching the same final position through a different action does not count. We screen object and robot state trajectories and manually review ambiguous successful trials in video, applying the same action criteria across models. CoMani-Seq reports SR for satisfaction of all final-goal predicates and progress rate (PR), the mean fraction of the ordered subtask prefix completed during execution. All rates are reported as percentages; a high PR need not imply final success.

V. EXPERIMENTS

A. Experimental Setup

Baselines and adaptation. We compare GraphPoint with ACT [1], DP [2], DP3 [4], PointPolicy [5], PointBridge [6], and $\pi_{0.5}$ [3]. ACT and DP use color images and robot state; DP3 uses scene point clouds and state. PointPolicy

and PointBridge use LIBERO entity points and gripper-point supervision. We adapt ACT, DP, DP3, and PointPolicy to multi-task instructions using frozen BGE embeddings of the full instruction, without role/action/modifier decomposition. ACT projects this embedding into an additional Transformer encoder token; PointPolicy appends a projected language token to its point-token sequence. DP and DP3 combine language and observation features through their existing FiLM conditioning paths. PointBridge retains its native language-token pathway: frozen MiniLM sentence embeddings pass through a learned MLP and precede observation tokens in the causal Transformer. $\pi_{0.5}$ retains its native image-text prefix conditioning and is fine-tuned with LoRA, without an added BGE encoder. For CoMani-Seq, atomic policies are reused without sequence-level training, receiving the active subtask instruction under a shared ES protocol; GraphPoint is also evaluated with SS.

Training and evaluation. All models are trained or fine-tuned on the first 10 demonstrations of each ID task for 30,000 steps with batch size 64 and AdamW; OOD tasks are excluded from training. We evaluate at 20 Hz over 30 trials per task on CoMani-Mod and CoMani-Act, and 10 trials per sequence on CoMani-Seq, using the splits in Sec. IV. Sequence rollouts allow 1,500 action steps after 30 settling steps and share initial-state indices across methods. We use the suite-specific metrics defined in Sec. IV: SR for CoMani-Mod, ACSR for CoMani-Act, and SR/PR for CoMani-Seq. Tables report task means and ID/OOD aggregates, with

TABLE II
CoMANI-MOD SUCCESS RATES. TASK IDS FOLLOW TABLE I.

Method	Mean SR		ID SR												OOD SR					
	ID	OOD	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
PointBridge	28.6	3.3	0.0	36.7	60.0	100.0	0.0	0.0	83.3	63.3	0.0	0.0	0.0	0.0	20.0	0.0	0.0	0.0	0.0	0.0
PointPolicy	21.4	15.0	3.3	36.7	66.7	50.0	0.0	0.0	16.7	0.0	23.3	0.0	13.3	46.7	10.0	0.0	0.0	6.7	0.0	73.3
DP3	73.6	57.2	73.3	100.0	100.0	100.0	53.3	100.0	0.0	80.0	80.0	86.7	26.7	83.3	86.7	63.3	60.0	93.3	16.7	23.3
ACT	20.0	7.2	33.3	20.0	6.7	23.3	13.3	56.7	10.0	3.3	63.3	0.0	0.0	10.0	30.0	0.0	0.0	13.3	0.0	0.0
DP	40.0	10.0	43.3	36.7	16.7	26.7	50.0	66.7	20.0	33.3	80.0	16.7	36.7	53.3	30.0	0.0	0.0	30.0	0.0	0.0
$\pi_{0.5}$	91.1	48.3	100.0	100.0	56.7	100.0	73.3	100.0	96.7	83.3	100.0	100.0	83.3	100.0	80.0	33.3	6.7	80.0	6.7	83.3
GraphPoint	96.4	75.6	100.0	100.0	100.0	86.7	93.3	100.0	100.0	80.0	100.0	100.0	96.7	100.0	100.0	63.3	73.3	76.7	73.3	66.7

TABLE III
CoMANI-ACT ACTION-CONSISTENT SUCCESS RATES (ACSR). TASK IDS FOLLOW TABLE I.

Method	Mean ACSR		ID ACSR							OOD ACSR		
	ID	OOD	0	1	2	3	4	5	6	7	8	9
PointPolicy	80.5	4.4	100.0	73.3	23.3	100.0	100.0	76.7	90.0	13.3	0.0	0.0
DP3	83.3	60.0	80.0	93.3	96.7	100.0	100.0	86.7	26.7	80.0	0.0	100.0
$\pi_{0.5}$												
GraphPoint	90.5	64.4	50.0	100.0	83.3	100.0	100.0	100.0	100.0	26.7	76.7	90.0



Fig. 4. Successful GraphPoint SS rollouts on CoMani-Seq. Rows show sequences containing 0, 1, and 2 OOD subtasks, respectively, from top to bottom. Each row proceeds from left to right, with 3 frames per subtask showing its start, execution, and completion. Upper-right values show predicted subtask progress. Apparent point-tracking mismatches are visualization artifacts, not actual tracking failures.

unfinished evaluations left blank.

We evaluate modifier and action composition, sequential skill reuse, and component contributions.

B. Can GraphPoint Generalize Across Modifiers?

Table II shows 96.4% ID and 75.6% OOD success for GraphPoint, exceeding the strongest baselines, $\pi_{0.5}$ on ID and DP3 on OOD, by 5.3 and 18.3 percentage points. ACT, DP, PointPolicy, and PointBridge have low ID success, so their OOD results reflect limitations in both task acquisition and generalization. GraphPoint achieves at least 63.3% on every OOD task, compared with DP3’s 16.7–93.3% range. For wine-bottle placement on the cabinet, it reaches 73.3% versus 16.7%; however, DP3 leads on placing cream cheese right of the cabinet (93.3% versus 76.7%). These results support broader held-out relation coverage, while leaving an ID–OOD gap.

C. Can GraphPoint Generalize Across Actions?

Table III evaluates action composition on CoMani-Act; $\pi_{0.5}$ results remain pending. PointPolicy reaches 80.5% ID and 4.4% OOD ACSR. GraphPoint reaches 90.5% ID and 64.4% OOD ACSR, compared with 83.3% and 60.0% for DP3. The modest average OOD advantage masks substantial differences across held-out tasks. GraphPoint transfers rotation from the stove knob to cream cheese with 76.7% ACSR, while DP3 obtains 0%. GraphPoint trails DP3 on putting cream cheese to the right of the stove, with 26.7% versus 80.0%, and on sweeping the bowl to the right of the plate, with 90.0% versus 100.0%. Thus, these results support cross-entity rotation transfer but do not establish a uniform advantage across action combinations.

D. Can GraphPoint Compose Atomic Skills?

Table IV evaluates atomic policies on six three-step sequences without sequence-level training. GraphPoint

TABLE IV

CoMANI-SEQ SUCCESS RATE (SR) AND PROGRESS RATE (PR). ENVIRONMENT-BASED SWITCHING (ES) USES GOAL SIGNALS; SELF-SWITCHING (SS) USES PREDICTED PROGRESS. MEAN GROUPS BY OOD COUNT; TASK IDS FOLLOW TABLE I.

Method	Switch	Mean								3 ID				2 ID + 1 OOD				2/1 ID + 1/2 OOD			
		All		0 OOD		1 OOD		2 OOD		0		1		2		3		4		5	
		SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR
PointPolicy	ES	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
DP3	ES	0.0	11.1	0.0	16.7	0.0	11.1	0.0	0.0	0.0	26.7	0.0	6.7	0.0	30.0	0.0	3.3	0.0	0.0	0.0	0.0
$\pi_{0.5}$	ES	0.0	25.0	0.0	28.3	0.0	26.7	0.0	13.3	0.0	33.3	0.0	23.3	0.0	33.3	0.0	33.3	0.0	13.3	0.0	13.3
GraphPoint	ES	51.7	79.4	75.0	95.0	50.0	81.1	10.0	43.3	60.0	90.0	90.0	100.0	30.0	76.7	30.0	70.0	90.0	96.7	10.0	43.3
GraphPoint	SS	38.3	78.3	65.0	90.0	26.7	77.8	20.0	56.7	90.0	96.7	40.0	83.3	20.0	76.7	10.0	73.3	50.0	83.3	20.0	56.7

TABLE V

CoMANI-MOD ABLATION SUCCESS RATES. TASK IDS FOLLOW TABLE I.

Variant	Mean SR		ID SR											OOD SR						
	ID	OOD	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
w/o RPC	90.6	44.4	100.0	96.7	100.0	100.0	66.7	96.7	96.7	83.3	93.3	100.0	70.0	83.3	20.0	90.0	80.0	6.7	10.0	60.0
Full connectivity	95.3	63.9	96.7	96.7	100.0	100.0	100.0	83.3	100.0	86.7	100.0	100.0	80.0	100.0	66.7	86.7	70.0	50.0	36.7	73.3
Merged entity point set	82.8	27.8	93.3	96.7	100.0	100.0	93.3	26.7	83.3	56.7	100.0	96.7	50.0	96.7	60.0	0.0	36.7	3.3	3.3	63.3
Absolute point coordinates	92.5	25.0	90.0	100.0	100.0	96.7	86.7	100.0	96.7	46.7	100.0	100.0	93.3	100.0	3.3	63.3	3.3	6.7	6.7	66.7
Delta-action output	95.0	73.3	96.7	100.0	100.0	100.0	100.0	96.7	80.0	86.7	100.0	100.0	80.0	100.0	96.7	86.7	76.7	76.7	50.0	53.3
GraphPoint	96.4	75.6	100.0	100.0	100.0	86.7	93.3	100.0	100.0	80.0	100.0	100.0	96.7	100.0	100.0	63.3	73.3	76.7	73.3	66.7

achieves 51.7% SR and 79.4% PR with ES, versus 38.3% and 78.3% with SS using patient/target progress summaries. The 13.33-point SR drop despite a 1.11-point PR decrease shows that similar partial progress does not guarantee full completion. With ES, PointPolicy, DP3, and $\pi_{0.5}$ all obtain 0% SR, with PR of 0.0%, 11.1%, and 25.0%, respectively. GraphPoint’s ES success decreases from 75.0% with no OOD subtasks to 50.0% with one and 10.0% with two. In the final task pair, changing only bowl-on-stove to the OOD bowl-right-of-stove subtask reduces SR from 90.0% to 10.0%, illustrating how atomic generalization limits sequence completion. Fig. 4 shows successful SS executions across all three OOD counts and their entity/progress transitions.

E. Which Components Matter?

Table V examines GraphPoint variants on CoMani-Mod. We remove RPC, replace chain attention with full connectivity, merge the entity graph into a single point set, use absolute point coordinates, or predict Cartesian delta actions instead of point trajectories.

Geometry and graph structure. Removing RPC reduces OOD success by 31.1 percentage points, compared with a 5.8-point ID decrease, consistent with its intended role in reducing dependence on detailed training geometry. Full graph connectivity retains 95.3% ID success but lowers OOD success to 63.9%, supporting patient-mediated interaction as a useful inductive bias. Merging all entity points yields 27.8% OOD success despite retaining 12 summary tokens in total. This variant jointly removes role embeddings, separate local entity groups, and chain connectivity, and reads shared summaries for progress; it therefore tests the structured encoder as a whole rather than any single component.

Coordinate and output representations. Using absolute point coordinates lowers OOD success to 25.0% while re-

taining 92.5% ID success, indicating that relative geometry is particularly useful for held-out combinations. Replacing point trajectories with Cartesian delta actions gives 95.0% ID and 73.3% OOD success, close to the full model. The observed gains are therefore more pronounced for relative coordinates, geometric augmentation, and role structure than for point output alone. These comparisons describe the reported checkpoint evaluations; repeated-training estimates are needed to assess the reliability of small differences.

VI. CONCLUSION

We introduced CoMani to isolate modifier, action-type, and sequential composition, and GraphPoint to connect semantic entity graphs with point-trajectory control. Results support role-structured interaction, TCP-relative geometry, and role-aware augmentation, while self-switching remains challenging. Evaluation is limited to simulation, 3-step sequences, and a small set of entities and modifiers. Occlusion can disrupt localization and tracking; upstream language parsing, localization, and segmentation add processing overhead and can propagate errors to control. Despite focusing on task-relevant entities, the system lacks explicit obstacle avoidance for surrounding objects. Future work will improve perception robustness, front-end efficiency, and obstacle-aware control. The point-input, point-output interface also provides a common geometric representation for exploring transfer from simulation and human videos, and across robot embodiments; these capabilities remain to be evaluated.

REFERENCES

- [1] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” *arXiv preprint arXiv:2304.13705*, 2023.
- [2] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

- [3] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [4] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations,” *arXiv preprint arXiv:2403.03954*, 2024.
- [5] S. Haldar and L. Pinto, “Point policy: Unifying observations and actions with key points for robot manipulation,” *arXiv preprint arXiv:2502.20391*, 2025.
- [6] S. Haldar, L. Johansmeier, L. Pinto, A. Gupta, D. Fox, Y. Narang, and A. Mandlekar, “Point bridge: 3d representations for cross domain policy learning,” *arXiv preprint arXiv:2601.16212*, 2026.
- [7] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [8] Y. Zhu, Z. Jiang, P. Stone, and Y. Zhu, “Learning generalizable manipulation policies with object-centric 3d representations,” *arXiv preprint arXiv:2310.14386*, 2023.
- [9] G. Chen, T. Cui, M. Wang, C. Yang, M. Hu, H. Lu, Y. Mu, Z. Peng, T. Zhou, X. Jiang, *et al.*, “Graphmimic: Graph-to-graphs generative modeling from videos for policy learning,” in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 1756–1768.
- [10] H. Qi, C. Chen, and H. Yang, “Compose by focus: Scene graph-based atomic skills,” *arXiv preprint arXiv:2509.16053*, 2025.
- [11] M. Shridhar, L. Manuelli, and D. Fox, “Cliport: What and where pathways for robotic manipulation,” in *Conference on robot learning*. pmlr, 2022, pp. 894–906.
- [12] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan, “Vima: Robot manipulation with multimodal prompts,” 2023.
- [13] J. A. Mendez, M. Hussing, M. Gummadi, and E. Eaton, “Composuite: A compositional reinforcement learning benchmark,” *arXiv preprint arXiv:2207.04136*, 2022.
- [14] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022.
- [15] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, “C-pack: Packed resources for general chinese embeddings,” in *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*, 2024, pp. 641–649.
- [16] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, *et al.*, “Sam 2: Segment anything in images and videos,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 28 085–28 128.
- [17] S. Jung, A. Zholus, M. Sundermeyer, C. Doersch, R. Goroshin, D. J. Tan, S. Chandar, R. Triebel, and F. Tombari, “Tapnext++: What’s next for tracking any point (tap)?” *arXiv preprint arXiv:2604.10582*, 2026.
- [18] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” *arXiv preprint arXiv:2210.02747*, 2022.
- [19] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, “Mimicgen: A data generation system for scalable robot learning using human demonstrations,” *arXiv preprint arXiv:2310.17596*, 2023.